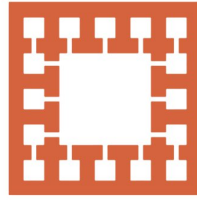


VARENDRA UNIVERSITY



বরেন্দ্র
বিশ্ববিদ্যালয়

Varendra University

Dept. of Computer Science & Engineering (CSE)

The Processor Status and the Flag Register (Chapter-5)

Course Title: **Microprocessor and Assembly Language**

Course No: **CSE 3205**

Prepared By:

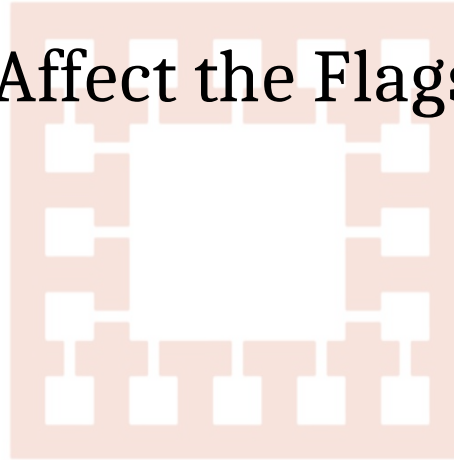
Md. Adnan Sami

Lecturer, Dept. of CSE, VU

Outline

1. Flag Register.
2. Overflow.
3. How Instructions Affect the Flags.
4. Overflow Math.
5. Practice Problem.

VARENDRA UNIVERSITY



বাবু
বিশ্ববিদ্যালয়

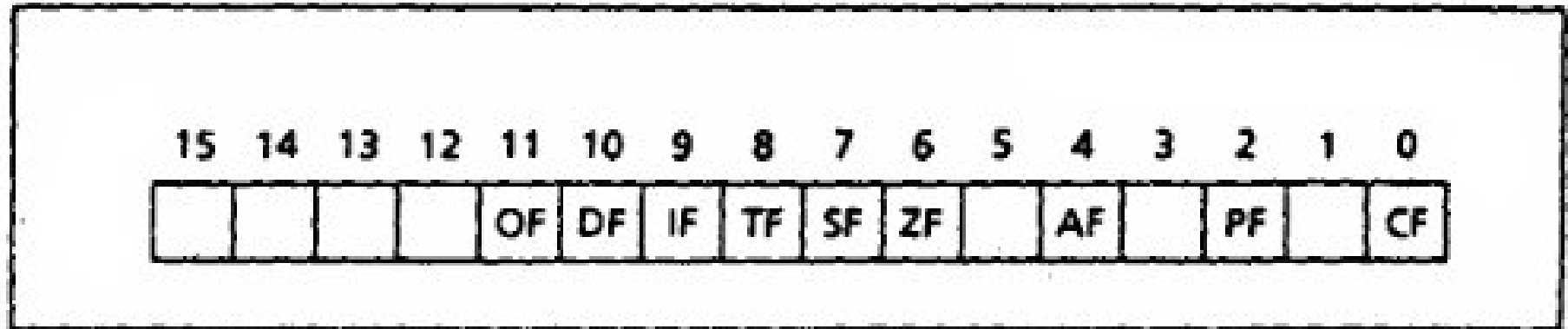
Flag Register

- For the 8086 processor, the processor state is implemented as nine individual bits called flags.
- Each decision made by the 8086 is based on the values of these flags.
- The flags are placed in the FLAG register and they are classified as either status flags or control flags.
- The status flags reflect the result of a computation.
- The control flags are used to enable or disable certain operations of the processor.

Flag Register

Flag register are two types,

1. Status Flag.
2. Control Flag.



- The status flags are located in bits 0, 2, 4, 6, 7, and 11.
- The control flags are located in bit 8, 9, and 10.

Flag Register

Table 5.1 Flag Names and Symbols

Status Flags

Bit	Name	Symbol
0	Carry flag	CF
2	Parity flag	PF
4	Auxiliary carry flag	AF
6	Zero flag	ZF
7	Sign flag	SF
11	Overflow flag	OF

Control Flags

Bit	Name	Symbol
8	Trap flag	TF
9	Interrupt flag	IF
10	Direction flag	DF

Flag Register

Status Flag:

The processor uses status flags to reflect the result of an operation.

Carry Flag (CF):

- CF = 1, if there is a carry out from the most significant bit (MSB) on addition, or there is a borrow into the MSB on subtraction. Otherwise, it is 0.
- CF is also affected by shift and rotate instructions.

Flag Register

Parity Flag (PF):

- PF = 1, if the low byte of a result has an **even number of one** bits (even parity).
- It is 0 if the low byte has odd parity.

For example,

If the result of a word addition is FFFEh (**1111 1111 1111 1110**), then the low byte contains 7 one bits, so PF = 0.

Auxiliary Flag (AF):

- AF = 1, if there is a carry out from bit 3 on addition, or a borrow into bit 3 on subtraction.
- AF is used in binary-coded decimal (BCD) operations.

Flag Register

Zero Flag (ZF):

- $ZF = 1$ for a zero result.
- $ZF = 0$ for a nonzero result.

Sign Flag (SF):

- $SF = 1$, if the MSB of a result is 1 (It means the result is negative)
- $SF = 0$, if the MSB is 0 .

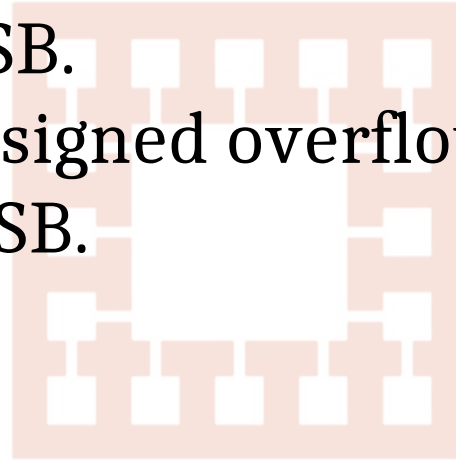
Overflow Flag (OF):

- $OF = 1$, if signed overflow occurred, otherwise it is 0.

Overflow

Unsigned Overflow:

- On addition, unsigned overflow occurs when there is a carry out of the MSB.
- On subtraction, unsigned overflow occurs when there is a borrow into the MSB.



বরেন্দ্র
বিশ্ববিদ্যালয়

Overflow

Signed Overflow:

- On addition of numbers with the same sign, signed overflow occurs when the sum has a different sign. **For example**, when we add 7FFFh and 7FFFh (two positive numbers), but got FFFEh (a negative result).
- Subtraction of numbers with different signs is like adding numbers of the same sign. For example, $A - (-B) = A + B$ and $-A - (+B) = -A + -B$. Signed overflow occurs if the result has a different sign than expected.
- In addition of numbers with different signs, overflow is impossible.

How Instructions Affect the Flags

<i>Instruction</i>	<i>Affects flags</i>
MOV/XCHG	none
ADD/SUB	all
INC/DEC	all except CF
NEG	all (CF = 1 unless result is 0, OF = 1 if word operand is 8000h, or byte operand is 80h)

विश्वविद्यालय

Overflow Math



Practice Problem

Example 5.1:

Example 5.1 ADD AX,BX, where AX contains FFFFh, BX contains FFFFh.

Solution:

$$\begin{array}{r} \text{FFFFh} \\ + \text{FFFFh} \\ \hline 1 \text{ FFFEh} \end{array}$$

The result stored in AX is FFFEh = 1111 1111 1111 1110.

SF = 1 because the msb is 1.

PF = 0 because there are 7 (odd number) of 1 bits in the low byte of the result.

ZF = 0 because the result is nonzero.

CF = 1 because there is a carry out of the msb on addition.

OF = 0 because the sign of the stored result is the same as that of the numbers being added (as a binary addition, there is a carry into the msb and also a carry out).

Practice Problem

Example 5.2:

Example 5.2 ADD AL,BL, where AL contains 80h, BL contains 80h.

Solution:

$$\begin{array}{r} 80h \\ + 80h \\ \hline 100h \end{array}$$

The result stored in AL is 00h.

SF = 0 because the msb is 0.

PF = 1 because all the bits in the result are 0.

ZF = 1 because the result is 0.

CF = 1 because there is a carry out of the msb on addition.

OF = 1 because the numbers being added are both negative, but the result is 0 (as a binary addition, there is no carry into the msb but there is a carry out).

Practice Problem

Example 5.3:

Example 5.3 SUB AX,BX, where AX contains 8000h and BX contains 0001h.

Solution:

$$\begin{array}{r} 8000h \\ - 0001h \\ \hline 7FFFh = 0111\ 1111\ 1111\ 1111 \end{array}$$

The result stored in AX is 7FFFh.

SF = 0 because the msb is 0.

PF = 1 because there are 8 (even number) one bits in the low byte of the result.

ZF = 0 because the result is nonzero.

CF = 0 because a smaller unsigned number is being subtracted from a larger one.

Now for OF. In a signed sense, we are subtracting a positive number from a negative one, which is like adding two negatives. Because the result is positive (the wrong sign), OF = 1.

Practice Problem

Example 5.4:

Example 5.4 INC AL, where AL contains FFh.

Solution:

$$\begin{array}{r} \text{FFh} \\ + \text{1h} \\ \hline \text{1 00h} \end{array}$$

The result stored in AL is 00h. SF = 0, PF = 1, ZF = 1. Even though there is a carry out, CF is unaffected by INC. This means that if CF = 0 before the execution of the instruction, CF will still be 0 afterward.

OF = 0 because numbers of unlike sign are being added (there is a carry into the msb and also a carry out).

Practice Problem

Example 5.5:

Example 5.5 `MOV AX, -5`

Solution: The result stored in AX is `-5 = FFFBh`.

None of the flags are affected by MOV.



Practice Problem

Example 5.6:

Example 5.6 NEG AX, where AX contains 8000h.

Solution:

$$\begin{array}{r} 8000\text{h} = 1000\ 0000\ 0000\ 0000 \\ \text{one's complement} = 0111\ 1111\ 1111\ 1111 \\ \qquad \qquad \qquad \qquad \qquad \qquad + 1 \\ \hline 1000\ 0000\ 0000\ 0000 = 8000\text{h} \end{array}$$

The result stored in AX is 8000h.

SF = 1, PF = 1, ZF = 0.

CF = 1, because for NEG CF is always 1 unless the result is 0.

OF = 1, because the result is 8000h; when a number is negated, we would expect a sign change, but because 8000h is its own two's complement, there is no sign change.

In the next section, we introduce a program that lets us see the actual setting of the flags.